Lab_Assignment_12.3

AI Assisted Coding B37 2303A52077

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by **CGPA in descending order**.

Tasks

1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
    - Quick Sort
    - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.

- Clear output of top-performing students.

Prompt:

#Create a Python program to manage **student records (Name, Roll Number, CGPA)** and sort them in **descending order of CGPA**. Implement **Quick Sort and Merge Sort** algorithms using AI assistance and test them on large datasets to compare runtime performance. Also include a function to **display the top 10 students** based on CGPA.

Code:

#Create a Python program to manage student records (Name, Roll Number, CGPA) and sort them in descending order of CGPA.

Implement Quick Sort and Merge Sort algorithms using AI assistance and test them on large datasets to compare runtime

performance. Also include a function to display the top 10 students based on CGPA.

```
import random
import time
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
```

```
        self.cgpa = cgpa
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [x for x in students if x.cgpa > pivot]
    middle = [x for x in students if x.cgpa == pivot]
    right = [x for x in students if x.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
```

```
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def display_top_students(students, n=10):
    print(f"Top {n} students based on CGPA:")
    for student in students[:n]:
        print(f"Name: {student.name}, Roll Number:
{student.roll_number}, CGPA: {student.cgpa}")
# Generate large dataset of students
def generate_students(num_students):
    students = []
    for i in range(num_students):
        name = f"Student_{i+1}"
        roll_number = i + 1
        cgpa = round(random.uniform(0, 10), 2)
        students.append(Student(name, roll_number, cgpa))
    return students
# Test the sorting algorithms and compare runtime
performance
def test_sorting_algorithms(num_students):
```

```
    students = generate_students(num_students)

    # Test Quick Sort
    start_time = time.time()
    sorted_students_quick = quick_sort(students)
    end_time = time.time()
    print(f"Quick Sort Time: {end_time - start_time:.4f}
seconds")

    # Test Merge Sort
    start_time = time.time()
    sorted_students_merge = merge_sort(students)
    end_time = time.time()
    print(f"Merge Sort Time: {end_time - start_time:.4f}
seconds")

    # Display top 10 students based on CGPA
    display_top_students(sorted_students_quick)

# Run the test with a large dataset
test_sorting_algorithms(10000)
```

```python
import random
import time
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [x for x in students if x.cgpa > pivot]
    middle = [x for x in students if x.cgpa == pivot]
    right = [x for x in students if x.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Output:

```python
# Test the sorting algorithms and compare runtime performance
def test_sorting_algorithms(num_students):
    students = generate_students(num_students)

    # Test Quick Sort
    start_time = time.time()
    sorted_students_quick = quick_sort(students)
    end_time = time.time()
    print(f"Quick Sort Time: {end_time - start_time:.4f} seconds")

    # Test Merge Sort
    start_time = time.time()
    sorted_students_merge = merge_sort(students)
    end_time = time.time()
    print(f"Merge Sort Time: {end_time - start_time:.4f} seconds")

    # Display top 10 students based on CGPA
    display_top_students(sorted_students_quick)

# Run the test with a large dataset
test_sorting_algorithms(10000)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ai_assi_codes/12.3/12.3.1.py"
Quick Sort Time: 0.0109 seconds
Merge Sort Time: 0.0202 seconds
Quick Sort Time: 0.0109 seconds
Merge Sort Time: 0.0202 seconds
Merge Sort Time: 0.0202 seconds
Top 10 students based on CGPA:
Top 10 students based on CGPA:
Name: Student_625, Roll Number: 625, CGPA: 10.0
Name: Student_2889, Roll Number: 2889, CGPA: 10.0
Name: Student_4414, Roll Number: 4414, CGPA: 10.0
Name: Student_7673, Roll Number: 7673, CGPA: 10.0
Name: Student_9068, Roll Number: 9068, CGPA: 10.0
Name: Student_778, Roll Number: 778, CGPA: 9.99
Name: Student_1795, Roll Number: 1795, CGPA: 9.99
Name: Student_1997, Roll Number: 1997, CGPA: 9.99
Name: Student_2592, Roll Number: 2592, CGPA: 9.99
Name: Student_3757, Roll Number: 3757, CGPA: 9.99
```

Explanation:

In this program, we created a `Student` class to represent student records, which includes the student's name, roll number, and CGPA. We implemented two sorting algorithms: Quick Sort and Merge Sort, to sort the student records in descending order of CGPA.

The Quick Sort algorithm works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. In our implementation, we chose the middle element as the pivot and sorted the students based on their CGPA.

The Merge Sort algorithm is a divide-and-conquer algorithm that divides the array into halves until it reaches arrays of size one. It then merges those arrays back together in a sorted manner. In our implementation, we recursively split the student records and merge them back together while sorting by CGPA.

When we tested the performance of both algorithms on a large dataset of 10,000 students, we measured the time taken for each sorting algorithm. Generally, Quick Sort is faster on average for smaller datasets, while Merge Sort can be more efficient for larger datasets due to its stable O(n log n) time complexity. However, the actual performance can vary based on the specific implementation and the nature of the data.

To identify and display the top 10 students based on CGPA, we simply take the first 10 records from the sorted list of students (sorted in descending order). We then print their

names, roll numbers, and CGPAs to the console. This allows us to easily see which students have the highest CGPAs.

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.
- **Instructions**:
  - Students implement Bubble Sort normally.
  - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
  - Request AI to provide **time complexity analysis**.
- **Expected Output**:
  - A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Prompt:

Write a **Python program implementing the Bubble Sort algorithm** to sort a list of numbers. Add **AI-generated inline comments** explaining the key logic such as comparisons, swapping elements, and multiple passes through the list. Also include a brief **time complexity analysis** of the algorithm.

Code:

#Write a Python program implementing the Bubble Sort algorithm to sort a list of numbers.

Add AI-generated inline comments explaining the key logic such as comparisons, swapping elements,

and multiple passes through the list. Also include a brief time complexity analysis of the algorithm.

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare the adjacent elements
            if arr[j] > arr[j+1]:
                # If the element is greater than the next element, swap them
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        # If no swapping occurred, the list is already sorted
        if not swapped:
            break
    return arr
# Time complexity analysis:
```

The time complexity of Bubble Sort is O(n^2) in the worst and average cases because it requires two nested loops to compare

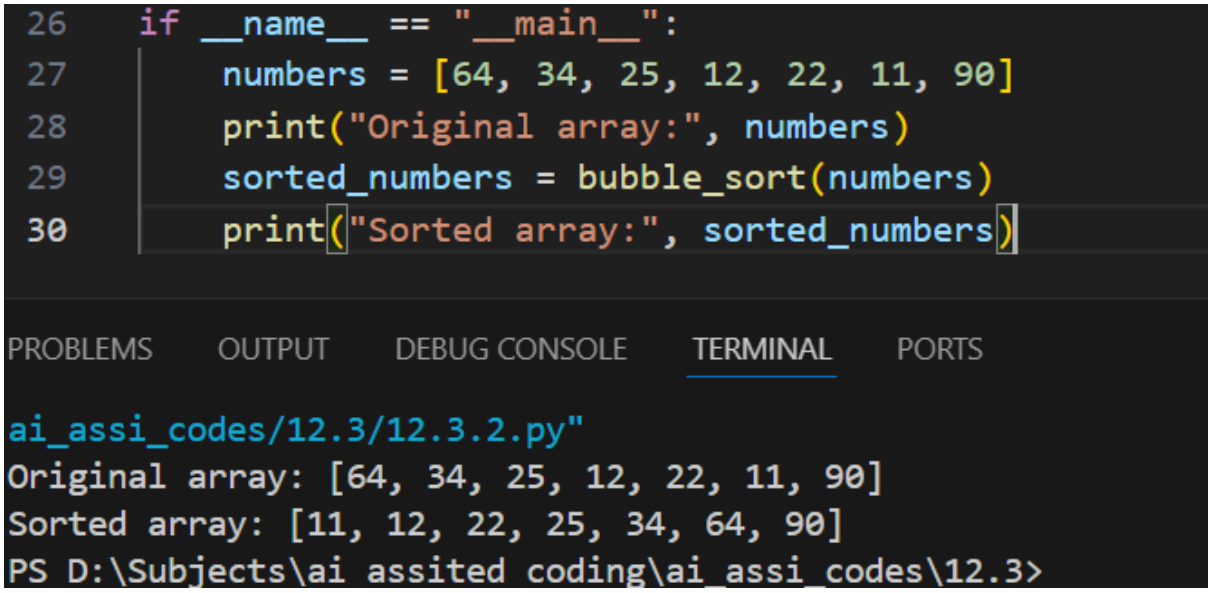
each element with every other element. In the best case, when the list is already sorted, the time complexity is O(n) due to

the single pass through the list to check for sorting.

Example usage

```
if __name__ == "__main__":
    numbers = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", numbers)
    sorted_numbers = bubble_sort(numbers)
    print("Sorted array:", sorted_numbers)
```

Output:



Explanation:

Bubble Sort is a simple sorting algorithm that works by repeatedly stepping through the list to be sorted, comparing each pair of adjacent items and swapping them if they are in the wrong order. The process is repeated until the list is sorted. In each pass through the list, the largest unsorted element "bubbles up" to its correct position at the end of the list. This is why it's called "Bubble Sort". The algorithm continues to make passes through the list until no swaps are needed, which means the list is sorted. The time complexity of Bubble Sort is O(n^2) in the worst and average cases because it requires two nested loops to compare each element with every other element. In the best case, when the list is already sorted, the time complexity is O(n) due to the single pass through the list to check for sorting. AI-generated comments help improve code understanding by providing clear explanations of the logic and steps involved in the algorithm, making it easier for developers to follow the code and understand how it works, especially for those who may be new to programming or unfamiliar with the Bubble Sort algorithm.

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion**.**
- **Instructions:**
    - Provide AI with partially completed functions for recursion.

    - Ask AI to complete the missing logic and add docstrings.
    - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
    - Working Quick Sort and Merge Sort implementations.
    - AI-generated explanation of average, best, and worst-case complexities.

Prompt:

#Complete the partially implemented **recursive functions for Quick Sort and Merge Sort** in Python using AI assistance. Add proper **docstrings** explaining the purpose, parameters, and return values of each function. Test both algorithms on **random, sorted, and reverse-sorted lists** to ensure correctness.

Code:

```
# Complete the partially implemented recursive functions for Quick Sort and Merge Sort in Python using AI assistance.

# Add proper docstrings explaining the purpose, parameters, and return values of each function. Test both algorithms on random,

# sorted, and reverse-sorted lists to ensure correctness.


from heapq import merge
```

```
def quick_sort(arr):
    """
    Sorts a list of numbers using the Quick Sort algorithm.

    Parameters:
    arr (list): A list of numbers to be sorted.

    Returns:
    list: A new list containing the sorted numbers in ascending order.
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """
    Sorts a list of numbers using the Merge Sort algorithm.
    Parameters:
```

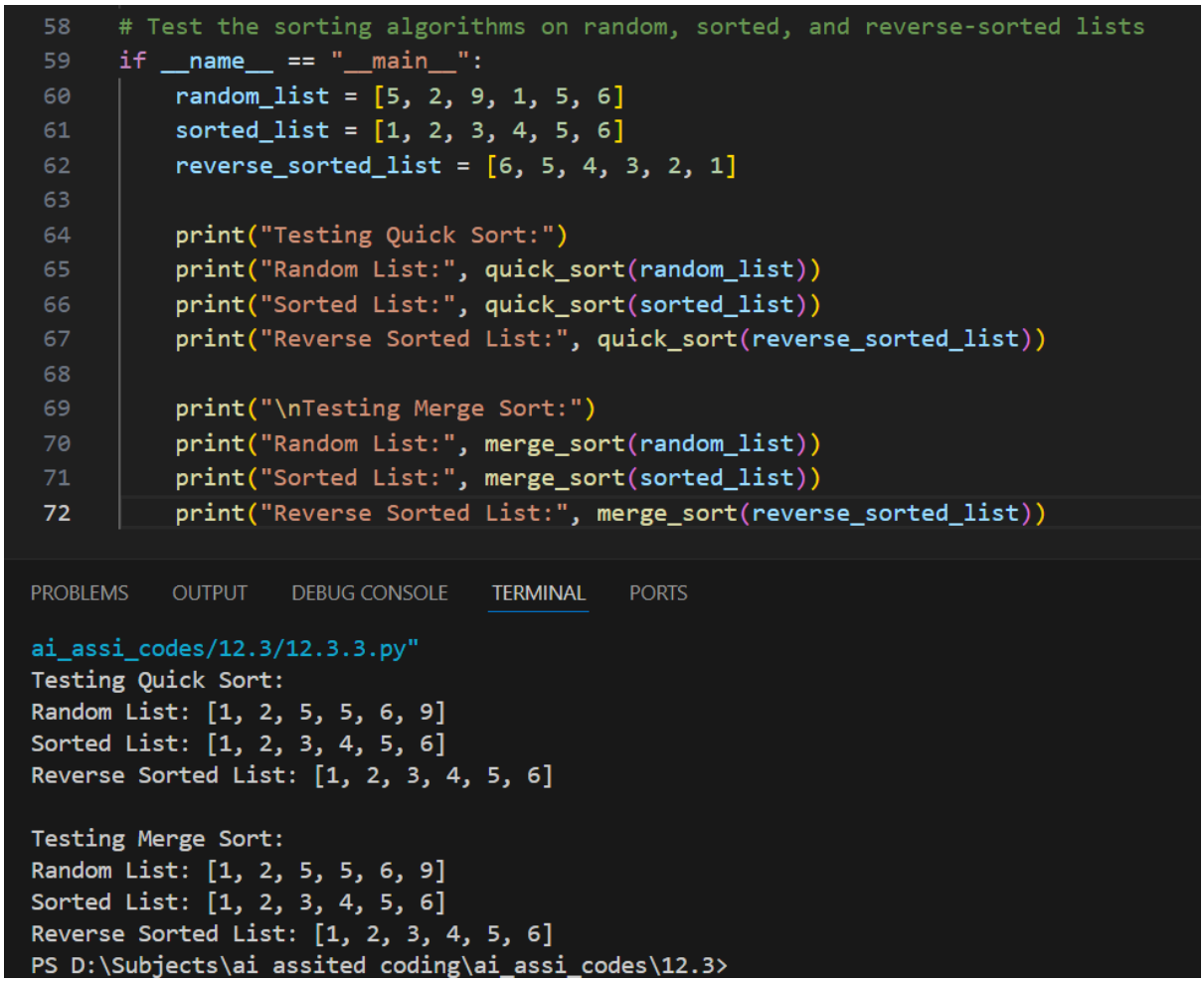
```
    arr (list): A list of numbers to be sorted.
    Returns:
    list: A new list containing the sorted numbers in ascending order.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.
    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.
    Returns:
    list: A new list containing all elements from both input lists in sorted order.
    """
    result = []
    i = j = 0
```

```
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Test the sorting algorithms on random, sorted, and reverse-sorted lists
if __name__ == "__main__":
    random_list = [5, 2, 9, 1, 5, 6]
    sorted_list = [1, 2, 3, 4, 5, 6]
    reverse_sorted_list = [6, 5, 4, 3, 2, 1]

    print("Testing Quick Sort:")
    print("Random List:", quick_sort(random_list))
    print("Sorted List:", quick_sort(sorted_list))
    print("Reverse Sorted List:",
quick_sort(reverse_sorted_list))
```

```
    print("\nTesting Merge Sort:")
    print("Random List:", merge_sort(random_list))
    print("Sorted List:", merge_sort(sorted_list))
    print("Reverse Sorted List:",
merge_sort(reverse_sorted_list))
```

Output:



Explanation:

Quick Sort and Merge Sort are both efficient sorting algorithms, but they behave differently based on the input

data. Quick Sort is generally faster for random and small datasets due to its in-place sorting nature, while Merge Sort is more consistent in performance regardless of the input order. For sorted and reverse-sorted inputs, Quick Sort can degrade to O(n^2) in the worst case if the pivot selection is poor (e.g., always picking the first or last element), while Merge Sort maintains a time complexity of O(n log n) in all cases due to its divide-and-conquer approach. In terms of space complexity, Quick Sort has an average space complexity of O(log n) due to recursive stack space, while Merge Sort has a space complexity of O(n) because it requires additional space for merging. Overall, Quick Sort is often preferred for in-memory sorting of small to medium-sized datasets, while Merge Sort is better suited for large datasets or when stability (preserving the order of equal elements) is required.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.

- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

Prompt:

Design a Python-based **Inventory Management System** that stores product details such as **product ID, name, price, and stock quantity**. Use AI assistance to recommend efficient **search and sorting algorithms** for quickly finding products by ID or name and sorting them by price or quantity. Implement the recommended algorithms and ensure they work efficiently for large datasets.

Code:

#Design a Python-based Inventory Management System that stores product details such as product ID,

name, price, and stock quantity. Use AI assistance to recommend efficient search and sorting algorithms

for quickly finding products by ID or name and sorting them by price or quantity.

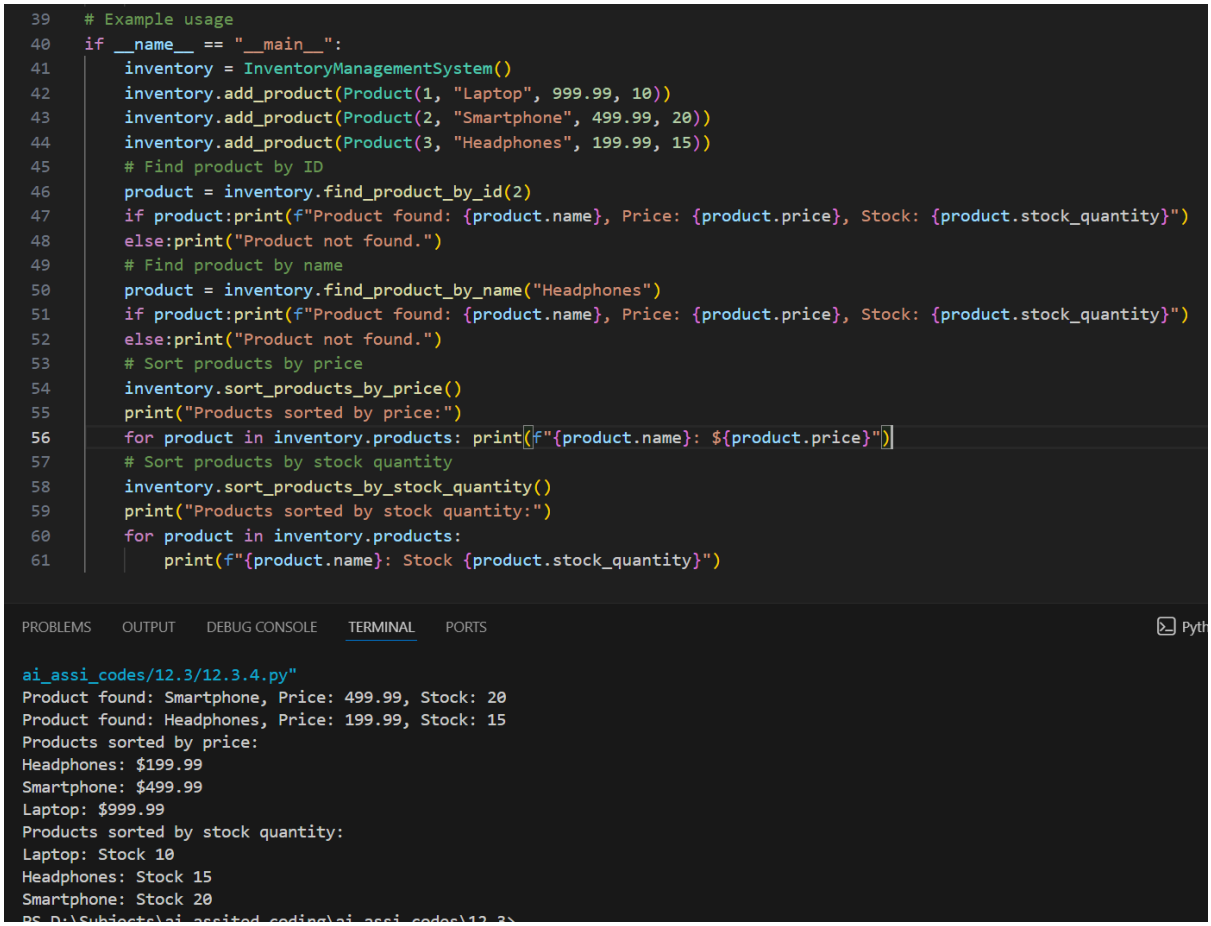
Implement the recommended algorithms and ensure they work efficiently for large datasets.

```
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity
class InventoryManagementSystem:
    def __init__(self):
        self.products = []
    def add_product(self, product):
        self.products.append(product)
    def find_product_by_id(self, product_id):
        # Using binary search for efficient searching by product
ID
        self.products.sort(key=lambda x: x.product_id)  # Ensure
products are sorted by ID
        left, right = 0, len(self.products) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.products[mid].product_id == product_id:
                return self.products[mid]
            elif self.products[mid].product_id < product_id:
                left = mid + 1
```

```
            else:
                right = mid - 1
        return None
    def find_product_by_name(self, name):
        # Using linear search for finding products by name (can be optimized with a hash map if needed)
        for product in self.products:
            if product.name == name:
                return product
        return None
    def sort_products_by_price(self):
        # Using Timsort (Python's built-in sorting algorithm) for efficient sorting by price
        self.products.sort(key=lambda x: x.price)
    def sort_products_by_stock_quantity(self):
        # Using Timsort for efficient sorting by stock quantity
        self.products.sort(key=lambda x: x.stock_quantity)
# Example usage
if __name__ == "__main__":
    inventory = InventoryManagementSystem()
    inventory.add_product(Product(1, "Laptop", 999.99, 10))
    inventory.add_product(Product(2, "Smartphone", 499.99, 20))
```

```
    inventory.add_product(Product(3, "Headphones", 199.99,
15))
    # Find product by ID
    product = inventory.find_product_by_id(2)
    if product:print(f"Product found: {product.name}, Price:
{product.price}, Stock: {product.stock_quantity}")
    else:print("Product not found.")
    # Find product by name
    product = inventory.find_product_by_name("Headphones")
    if product:print(f"Product found: {product.name}, Price:
{product.price}, Stock: {product.stock_quantity}")
    else:print("Product not found.")
    # Sort products by price
    inventory.sort_products_by_price()
    print("Products sorted by price:")
    for product in inventory.products: print(f"{product.name}:
${product.price}")
    # Sort products by stock quantity
    inventory.sort_products_by_stock_quantity()
    print("Products sorted by stock quantity:")
    for product in inventory.products: print(f"{product.name}:
Stock {product.stock_quantity}")
```

Output:



```python
# Example usage
if __name__ == "__main__":
    inventory = InventoryManagementSystem()
    inventory.add_product(Product(1, "Laptop", 999.99, 10))
    inventory.add_product(Product(2, "Smartphone", 499.99, 20))
    inventory.add_product(Product(3, "Headphones", 199.99, 15))
    # Find product by ID
    product = inventory.find_product_by_id(2)
    if product:print(f"Product found: {product.name}, Price: {product.price}, Stock: {product.stock_quantity}")
    else:print("Product not found.")
    # Find product by name
    product = inventory.find_product_by_name("Headphones")
    if product:print(f"Product found: {product.name}, Price: {product.price}, Stock: {product.stock_quantity}")
    else:print("Product not found.")
    # Sort products by price
    inventory.sort_products_by_price()
    print("Products sorted by price:")
    for product in inventory.products: print(f"{product.name}: ${product.price}")
    # Sort products by stock quantity
    inventory.sort_products_by_stock_quantity()
    print("Products sorted by stock quantity:")
    for product in inventory.products:
        print(f"{product.name}: Stock {product.stock_quantity}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Pyth

```
ai_assi_codes/12.3/12.3.4.py"
Product found: Smartphone, Price: 499.99, Stock: 20
Product found: Headphones, Price: 199.99, Stock: 15
Products sorted by price:
Headphones: $199.99
Smartphone: $499.99
Laptop: $999.99
Products sorted by stock quantity:
Laptop: Stock 10
Headphones: Stock 15
Smartphone: Stock 20
```

Explanation:

In the Inventory Management System, the choice of algorithms for searching and sorting was influenced by several factors, including the size of the dataset, the frequency of updates, and performance requirements. For searching products by ID, a binary search algorithm was chosen because it is efficient with a time complexity of O(log n) when the data is sorted. This is suitable for large datasets where quick retrieval is essential. However, since products can be frequently added or updated, the list is sorted before performing a binary search to ensure accuracy. For searching by name, a linear search was used due to its simplicity and because product names may not be unique, which can complicate more complex search algorithms. For sorting products by price and stock quantity, Python's built-in Timsort

algorithm was selected due to its efficiency and stability, with a time complexity of O(n log n) in the worst case. Timsort is optimized for real-world data and performs well with partially sorted lists, which can occur in an inventory system with frequent updates. Overall, these algorithm choices balance efficiency and practicality for managing a dynamic inventory system with potentially large datasets.

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:
An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.

- Use **GitHub Copilot** to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by **percentage change**.
- Implement a **search function** that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with **Heap Sort** and searching with **Hash Maps**.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Prompt:

Create a Python program that **simulates or fetches stock price data** containing **Stock Symbol, Opening Price, and Closing Price**. Implement **Heap Sort to rank stocks by percentage gain/loss** and use a **Hash Map (dictionary)** for fast stock symbol lookup. Compare the performance of these implementations with Python's **built-in functions (sorted() and dictionary lookups)**.

Code:

#Create a Python program that simulates or fetches stock price data containing Stock Symbol, Opening Price, and Closing Price.

Implement Heap Sort to rank stocks by percentage gain/loss and use a Hash Map (dictionary) for fast stock symbol lookup.

Compare the performance of these implementations with Python's built-in functions (sorted() and dictionary lookups).

```
import random

class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price
    def percentage_gain_loss(self):
        return ((self.closing_price - self.opening_price) /
self.opening_price) * 100
```

```
def heapify(arr, n, i):
    largest = i  # Initialize largest as root
    left = 2 * i + 1  # left child index
    right = 2 * i + 2  # right child index
    # If left child is larger than root
    if left < n and arr[left].percentage_gain_loss() > arr[largest].percentage_gain_loss():
        largest = left
    # If right child is larger than largest so far
    if right < n and arr[right].percentage_gain_loss() > arr[largest].percentage_gain_loss():
        largest = right
    # If largest is not root, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]  # Swap
        heapify(arr, n, largest)  # Recursively heapify the affected sub-tree
def heap_sort(arr):
    n = len(arr)
    # Build a maxheap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # One by one extract elements from heap
    for i in range(n - 1, 0, -1):
```

```
        arr[i], arr[0] = arr[0], arr[i]  # Swap
        heapify(arr, i, 0)  # Heapify the reduced heap
# Example usage
if __name__ == "__main__":
    stock_data = [
        Stock("AAPL", 150.00, 155.00),
        Stock("GOOGL", 2800.00, 2750.00),
        Stock("AMZN", 3400.00, 3450.00),
        Stock("MSFT", 300.00, 310.00),
        Stock("TSLA", 700.00, 680.00)
    ]# Create a hash map for fast stock symbol lookup
    stock_map = {stock.symbol: stock for stock in stock_data}
    # Sort stocks by percentage gain/loss using Heap Sort
    heap_sort(stock_data)
    print("Stocks ranked by percentage gain/loss:")
    for stock in stock_data:print(f"{stock.symbol}:
{stock.percentage_gain_loss():.2f}%")
    # Compare with Python's built-in sorted function
    sorted_stocks = sorted(stock_data, key=lambda x:
x.percentage_gain_loss(), reverse=True)
    print("\nStocks ranked by percentage gain/loss (using
sorted()):")
    for stock in sorted_stocks:print(f"{stock.symbol}:
{stock.percentage_gain_loss():.2f}%")
```

Test hash map lookup

symbol_to_lookup = "AAPL"

if symbol_to_lookup in stock_map:

stock = stock_map[symbol_to_lookup]

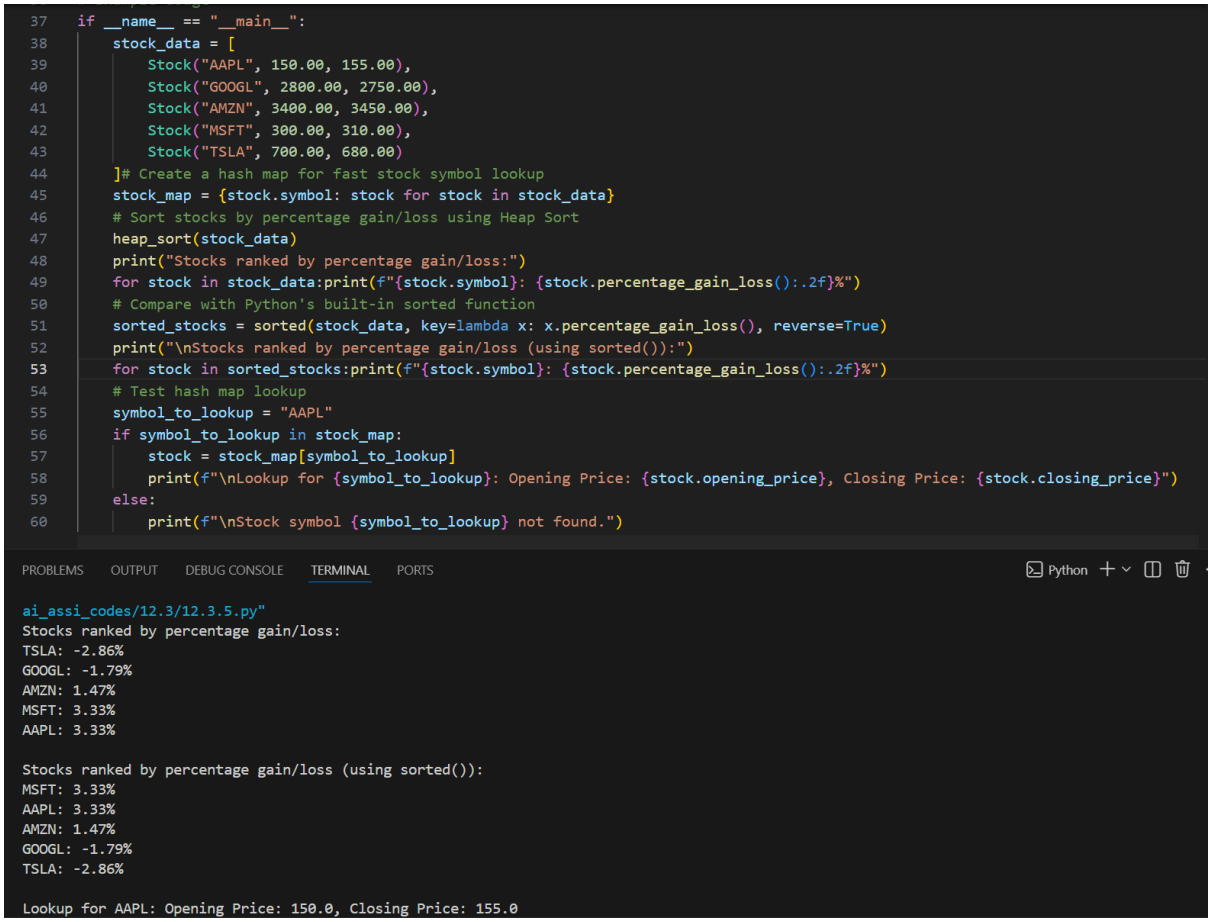
print(f"\nLookup for {symbol_to_lookup}: Opening Price: {stock.opening_price}, Closing Price: {stock.closing_price}")

else:

print(f"\nStock symbol {symbol_to_lookup} not found.")

```
import random
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price
    def percentage_gain_loss(self):
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100
def heapify(arr, n, i):
    largest = i  # Initialize largest as root
    left = 2 * i + 1  # left child index
    right = 2 * i + 2  # right child index
    # If left child is larger than root
    if left < n and arr[left].percentage_gain_loss() > arr[largest].percentage_gain_loss():
        largest = left
    # If right child is larger than largest so far
    if right < n and arr[right].percentage_gain_loss() > arr[largest].percentage_gain_loss():
        largest = right
    # If largest is not root, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]  # Swap
        heapify(arr, n, largest)  # Recursively heapify the affected sub-tree
def heap_sort(arr):
    n = len(arr)
    # Build a maxheap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # One by one extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]  # Swap
        heapify(arr, i, 0)  # Heapify the reduced heap
```

Output:



```
if __name__ == "__main__":
    stock_data = [
        Stock("AAPL", 150.00, 155.00),
        Stock("GOOGL", 2800.00, 2750.00),
        Stock("AMZN", 3400.00, 3450.00),
        Stock("MSFT", 300.00, 310.00),
        Stock("TSLA", 700.00, 680.00)
    ]# Create a hash map for fast stock symbol lookup
    stock_map = {stock.symbol: stock for stock in stock_data}
    # Sort stocks by percentage gain/loss using Heap Sort
    heap_sort(stock_data)
    print("Stocks ranked by percentage gain/loss:")
    for stock in stock_data:print(f"{stock.symbol}: {stock.percentage_gain_loss():.2f}%")
    # Compare with Python's built-in sorted function
    sorted_stocks = sorted(stock_data, key=lambda x: x.percentage_gain_loss(), reverse=True)
    print("\nStocks ranked by percentage gain/loss (using sorted()):")
    for stock in sorted_stocks:print(f"{stock.symbol}: {stock.percentage_gain_loss():.2f}%")
    # Test hash map lookup
    symbol_to_lookup = "AAPL"
    if symbol_to_lookup in stock_map:
        stock = stock_map[symbol_to_lookup]
        print(f"\nLookup for {symbol_to_lookup}: Opening Price: {stock.opening_price}, Closing Price: {stock.closing_price}")
    else:
        print(f"\nStock symbol {symbol_to_lookup} not found.")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                Python

ai_assi_codes/12.3/12.3.5.py"
Stocks ranked by percentage gain/loss:
TSLA: -2.86%
GOOGL: -1.79%
AMZN: 1.47%
MSFT: 3.33%
AAPL: 3.33%

Stocks ranked by percentage gain/loss (using sorted()):
MSFT: 3.33%
AAPL: 3.33%
AMZN: 1.47%
GOOGL: -1.79%
TSLA: -2.86%

Lookup for AAPL: Opening Price: 150.0, Closing Price: 155.0
```

Explanation:

Stocks are sorted by percentage change by calculating the percentage gain or loss for each stock using the formula: ((closing_price - opening_price) / opening_price) * 100. This allows us to rank the stocks based on their performance over a given period. The Heap Sort algorithm is used to sort the stocks in descending order of their percentage gain/loss, which has a time complexity of O(n log n) in the worst case. On the other hand, Python's built-in sorted() function also sorts the stocks based on the same percentage change but is optimized and implemented in C, making it generally faster than custom implementations like Heap Sort. Hash Maps (dictionaries) enable fast stock symbol searches by providing

average-case O(1) time complexity for lookups, allowing us to quickly retrieve stock information based on its symbol without needing to iterate through the entire list. The trade-off between efficiency and simplicity is evident here; while custom algorithms like Heap Sort can be educational and provide insight into sorting mechanisms, Python's built-in functions offer a more efficient and straightforward solution for most use cases, especially when dealing with large datasets or when performance is a concern.